

The Flow Compiler Architecture: Portable Code Generation, Algebraic Effects, and a Self-Hosting Bootstrap

Abhishek Shivakumar¹

¹Independent research , <https://github.com/floooooooooooooow/flow>

August 2026

Abstract

Flow is a statically typed systems language whose design thesis is that programs should describe *how systems evolve through time*, rather than only sequences of imperative instructions. Realizing that thesis requires a compiler that is simultaneously (i) expressive enough for algebraic effects, domain surface languages, and numerical computing, and (ii) pragmatic enough to emit portable native code without mandating a heavyweight LLVM dependency for the default path.

This paper presents the architecture of the Flow compilation toolchain as implemented in the open-source Flow repository. We describe a *dual-host* design: a production Python frontend that drives the full language surface through a multi-stage pipeline (surface expansion, module resolution, parsing, type checking, monomorphization, and multi-backend emission), and a Flow-written Stage-A compiler (`flowc`) that is progressively closing a classical self-hosting bootstrap loop against a C backend. We detail the dominant portable backend—direct translation of typed ASTs to C with an effect-handler runtime encoded as dynamically scoped vttables—and situate secondary targets (MLIR/LLVM, WebAssembly via Emscripten, Metal fill shaders, and JavaScript/pretty-print emitters in Stage A). We characterize surface-language expanders (dynamics `dsys`, fill shaders) as deliberate *pre-parse* transformations that keep the core grammar stable while enabling domain-specific authoring. Finally, we document the bootstrap ladder (Gen0 host emit → Gen1 self-emit → Gen2 fixed-point comparison), honest capability boundaries, and the engineering invariants that govern further self-hosting work.

Keywords: compilers, algebraic effects, C code generation, self-hosting, monomorphization, dynamical systems DSL, bootstrap

1 Introduction

The engineering workflow for dynamical, control, signal-processing, and scientific software is historically fragmented: models are written in one tooling stack, analysis in another, and deployment artifacts in a third. Flow [1] is an attempt to collapse that fragmentation by treating *evolution*—continuous or discrete change of explicit state under explicit timing and constraints—as a first-class programming abstraction, while retaining a conventional, statically typed general-purpose core.

A language with that ambition places unusual demands on its compiler. Algebraic effects must lower to a predictable ABI. Domain surface syntax for linear systems and shaders must not permanently enlarge the core grammar. Generics must disappear before code generation. The default backend must remain portable: a C host toolchain (Clang or GCC) should suffice for native binaries, with MLIR/LLVM and GPU paths available as opt-in specialization rather than mandatory infrastructure.

This paper is an architectural description of the Flow compiler as it exists in the public repository circa August 2026. It is intentionally calibrated: features that are library-level (e.g. forward-mode automatic differentiation via dual numbers) are not misrepresented as compiler passes; Stage-A self-hosting is reported as a *partial* fixed-point on a language subset, not as full replacement of the production host.

Contributions.

1. A precise account of Flow’s dual-host compilation architecture: production Python pipeline and Flow-written Stage-A bootstrap (`flowc`).
2. A description of the portable C backend’s effect-handler lowering (handler vtables, dynamic scoping, capability installation).
3. A characterization of surface-language expanders as architecture, not ad hoc string rewriting.
4. Documentation of the self-hosting bootstrap ladder and its current fixed-point evidence.

Non-goals. We do not claim a formal semantics for the full language, nor do we present new algorithmic results in parsing or optimization. Performance numbers cited from the repository’s benchmark suite are measurements of emitted C versus hand-written C under matched flags; they are context for the architecture, not the primary claim of this paper.

2 Design goals and constraints

Flow’s compiler architecture is shaped by five constraints that recur throughout the implementation.

Definition 1 (Portable default). *The default compile path must produce a native binary given only a C compiler and the Flow sources. LLVM/MLIR may accelerate or specialize, but must not be required for `flow run` on ordinary programs.*

Definition 2 (Effects as capabilities). *Side effects that would otherwise be ambient (logging, I/O, inventory, async delay) are expressed as effect operations; concrete behaviour is supplied by capabilities installed under `handle ... with ...` dynamic scopes [4].*

Domain vocabularies (linear dynamics blocks, fill shaders) may be authored in dedicated surface syntax, but enter the compiler via expanders that emit ordinary Flow before the recursive-descent parser runs.

Definition 4 (Honest subset boundaries). *The self-hosted Stage-A compiler documents what it accepts. Features required by the production language but absent from Stage-A remain on the Python host until the subset gap is closed deliberately.*

Definition 5 (Model-as-program). *Where feasible, analysis and deployment share one artifact: a Flow source that both declares a plant (or shader, or effectful service) and compiles to the executable that runs it [1].*

These definitions are operational: they explain why the pipeline places expanders before parsing, why C is the default IR-of-choice for shipping, and why `flowc` and `src/flow/*.py` coexist during bootstrap.

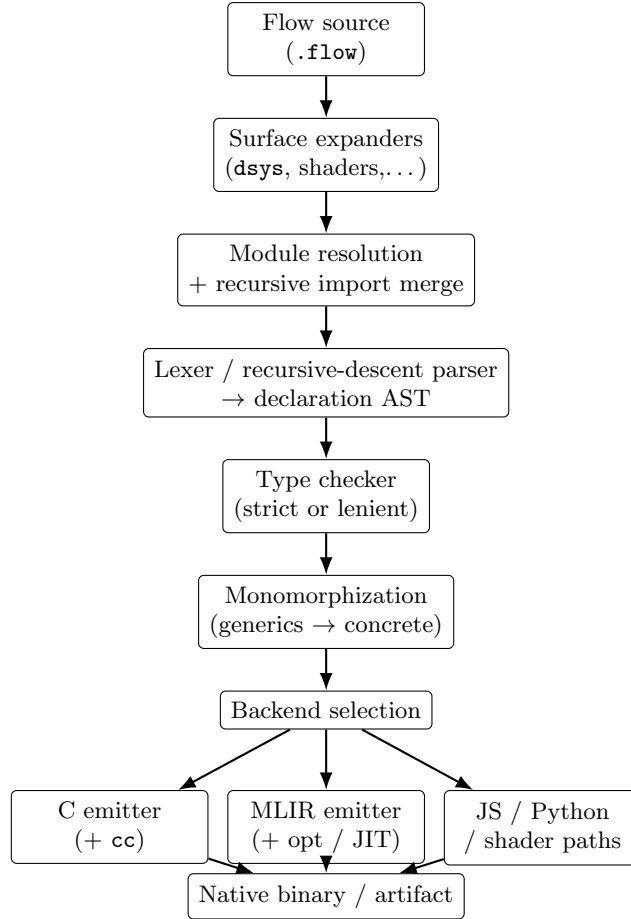


Figure 1: Production Flow compilation pipeline. Surface expanders rewrite domain syntax into core Flow before parsing. Generics are eliminated by monomorphization prior to backend emission. The portable default is C; MLIR and specialized paths are opt-in.

3 System overview

At repository scale (tracked sources on the order of 1.6×10^5 physical lines, of which roughly 7.3×10^4 are Flow and 3.4×10^4 are the Python compiler), the toolchain comprises:

- **CLI host** — the `./flow` shell entry point, which configures `PYTHONPATH`, discovers optional LLVM tools, and dispatches `run`, `compile`, `graphics`, `audio`, `shader`, and `test` commands.
- **Production compiler** — Python modules under `src/flow/`, approximately forty modules spanning parser, type checker, monomorphizer, C/MLIR/JS/Python emitters, LSP, and DSL expanders.
- **Stage-A compiler (flowc)** — Flow sources under `compiler/src/`, emitting C (and optionally JS or pretty-printed Flow) for a documented subset, with self-emit scripts establishing a Gen1/Gen2 object fixed point.
- **Runtime and stdlib** — C/Objective-C runtimes (graphics, GPU memory, HTTP natives) and Flow libraries (effects, autodiff, dynamics, NN, audio, memory).

Figure 1 summarizes the production path.

4 Production frontend

4.1 Module resolution

Compilation begins at a root file. The module resolver recursively loads imports (`import "path.flow"`), tracks visited files to break cycles, and merges exported symbols into a single declaration list for downstream passes [2]. Visibility is export-based: symbols are module-private by default.

Critically, resolution is also the hook point for surface dialects. Before a file is handed to the parser, the resolver may:

- detect and expand dynamics DSL blocks (`dsys`, `sense`, `ga evolve`, `closed`, `represent linear`, ...) into calls against `lib/stdlib/dynamics`;
- recognize fill-shader modules (`shader fill`) and divert them to the shader compilation path (or install a host stub so corpus transpile checks remain well-defined).

This ordering—expand, then parse—preserves a single recursive-descent grammar for the core language while still allowing domain-shaped authoring.

4.2 Lexer and parser

The production lexer/parser (`parser.py`) builds a typed AST of declarations and statements: functions, structs, effects, capabilities, enums, traits/impls, constants, and the usual structured control flow (`if/while/for ... in ... to ...`, `match`, `handle`).

Syntactic priorities of the living language include:

- explicit mutability (`let` vs. `let mut`);
- explicit types on bindings and signatures;
- algebraic effect and capability declarations;
- FFI via `extern { ... }` blocks;
- postfix chaining sufficient for systems libraries (indexing, field access, method calls).

4.3 Type checking

The `TypeChecker` walks the merged declaration list under strict mode by default (lenient mode demotes errors to warnings for transitional corpora). Checking covers name resolution, arity, return consistency, and the bulk of structural typing obligations expected of a C-emitting systems language. The checker is a semantic gate before monomorphization: generics remain in the AST until their uses are known.

4.4 Monomorphization

Flow generics are compiled by specialization, in the style of Rust’s monomorphization rather than Java-style erasure [10]. A generic function or struct

```
struct Option<T> { has_value: bool, value: T }  
function identity<T>(x: T) → T { return x }
```

is replaced, for each concrete instantiation site, by specialized clones (`Option_i32`, `identity_i32`, ...). The trade-off is the usual one: zero runtime dispatch cost and full static typing, at the expense of code size. After this pass, backends may assume a monomorphic program.

4.5 Mode guards

Declarations may carry attributes that restrict them to particular execution modes (`@only(jit)`, `@guard(c)`, `hot-reload`, ...). The transpiler filters the declaration list according to the active backend and CLI flags before emission, allowing a single source file to contain compile-time and JIT-only variants without polluting the default binary.

5 The portable C backend

5.1 Translation strategy

The C generator (`c_generator.py`) is the workhorse backend. It emits a single translation unit (or a small set of units linked with runtime objects) that a host `cc` can compile. Types map in the obvious way (`i32`→`int32_t`, `f64`→`double`, `ptr<T>`→`T*`, structs to C structs, arrays to C arrays or explicit buffers depending on shape).

Control flow lowers to C structured statements. Closures and higher-order patterns that the language admits are compiled with explicit environment representations where needed; the generator tracks nesting depth relative to `handle` blocks so that escaping closures do not accidentally capture transient handler state.

5.2 Algebraic effects as vtables

Flow’s algebraic effects are the clearest linguistic differentiator relative to C, Rust, Zig, and Mojo in the project’s own comparison notes [9]. The C lowering is deliberately simple and ABI-shaped:

1. Each **effect** `E { op(...) -> T, ... }` becomes a C handler struct of function pointers (a vtable) plus a thread-oriented current-handler pointer.
2. Each capability supplies concrete function implementations and a static vtable instance.
3. A `handle E with Cap { ... }` block saves the previous handler pointer, installs `Cap`’s vtable, executes the body, and restores the previous handler on exit (including nested handles).
4. Effect operations compile to calls through the current handler; absent a handler, operations degrade to defined no-ops / zero returns rather than aborting—a pragmatic choice for demos and partial stacks.

Remark 1 (Tail-resumptive MVP). The shipped effect system is *tail-resumptive*: operations return to the call site. There is no general delimited continuation, fiber scheduler, or work-stealing runtime in the default lowering. Asynchronous APIs are therefore modelled as effects with simulated or blocking capabilities, not as `async/await` desugaring [5].

This design keeps effectful Flow programs readable as ordinary C with an explicit capability ABI, and makes handler swapping (production vs. test backends) a matter of installing different vtables around the same call graph [4].

5.3 Linking and runtimes

Native features beyond ISO C—macOS/Linux graphics, Metal GPU memory, audio device I/O, HTTP natives in registry packages—are linked as separate runtime objects. The CLI selects which runtime objects to include based on imports and command mode (`flow run`, `flow gfx`, `flow audio`, ...). The architectural invariant is that the *language* backend remains C; platform capability arrives as libraries, not as a second IR.

6 Secondary backends and special paths

6.1 MLIR / LLVM

An MLIR generator lowers Flow ASTs to MLIR dialects (structured control flow, memrefs, optional GPU dialect emission). Optimizations may be applied via `mlir-opt`; JIT and hot-reload paths compile through LLVM tooling when present. This path is important for experimentation and GPU dialect work, but it is not the portability baseline.

6.2 WebAssembly

WASM is obtained by composing the C backend with Emscripten (`emcc`) [8]:

$$\text{Flow} \xrightarrow{\text{C backend}} \text{C} \xrightarrow{\text{emcc}} \text{.wasm} + \text{JS glue}.$$

There is no Flow-in-WASM self-hosted toolchain; browser delivery reuses the same portable C semantics as native builds.

6.3 Fill shaders

Fill-shader modules are a separate surface language compiled by `shader_codegen` / `./flow shader` to Metal fragment entry points. They are intentionally not forced through the host Flow C pipeline as ordinary functions; the module resolver isolates them so the core language does not absorb shading grammar.

6.4 Automatic differentiation

Forward-mode AD ships as library dual numbers; reverse-mode helpers and external gradient codegen tools exist as prototypes [7]. They are *not* currently a mandatory compiler IR pass. Architecturally this matters: AD can deepen into a transformation pass once a stable typed IR and mutation story for tapes exist; until then, the compiler treats AD as library and tooling layered on the same AST/C pipeline.

7 Surface languages as architectural components

7.1 Dynamics DSL (dsys)

The dynamics surface lets authors declare linear plants, horizons, sensing predicates (controllability, spectral radius), genetic-algorithm gain search, and closed-loop certificates without writing matrix boilerplate by hand. Expansion injects setup code into `main` and binds analysis results as ordinary locals [6]. Continuous plants are Euler-discretized for the present analysis envelope (notably small SISO LTI systems).

```
dsys plant {  
  discrete dt 0.1 n 2 m 1 p 1  
  A 1.0 0.1 0.0 1.0  
  B 0.0 0.1  
  C 1.0 0.0  
}  
sense on plant { controllable → ok spectral → rho }
```

The architectural claim is not that the expander is a full hybrid-systems compiler; it is that *domain syntax hooks at resolve-time* are the supported extension mechanism while the core grammar and C backend remain stable.

7.2 Evolution-oriented core syntax

Beyond `dsys`, the language vision includes `flow` blocks with `evolves as` dynamics and related timing/constraint forms. Where implemented, these participate in the same resolve/-parse/check/emit chain; where still aspirational, they are tracked as roadmap work rather than silent compiler magic [1].

8 Self-hosting: Stage-A flowc

8.1 Motivation

Long-term, Flow aims to retire Python from the compile critical path: dogfooding, a single toolchain story for users, and a subset forced by systems needs rather than by host-language convenience [3]. Tooling (wiki build, some LSP glue, benches) may remain Python longer; compilation should not.

8.2 Stage-A modules

`flowc` is structured as cooperating Flow modules with an index-based AST arena (children and sibling chains stored as integer indices into a node buffer—friendly to C emission and explicit memory management):

Module	File	Role
<code>token/lexer</code>	<code>token.flow, lexer.flow</code>	Lexical analysis
<code>ast</code>	<code>ast.flow</code>	Tagged AST arena
<code>parser</code>	<code>parser.flow</code>	Recursive-descent subset parser
<code>typecheck</code>	<code>typecheck.flow</code>	Lightweight name/semantic checks
<code>resolve</code>	<code>resolve.flow</code>	Multi-file import + bundle emit
<code>cgen</code>	<code>cgen.flow</code>	Stage-A AST→C buffer emitter
<code>jsgen/fmt</code>	<code>jsgen.flow, fmt.flow</code>	JS / pretty-print emitters
<code>driver/main</code>	<code>driver.flow, main.flow</code>	CLI / smoke / env-gated emit

8.3 Bootstrap ladder

The classical ladder is instantiated concretely:

1. **Gen0.** The Python host compiles `compiler/src/*.flow` to C objects and links a driver (`flowc_frontend.o` plus C or Flow driver).
2. **Gen1.** That driver re-emits the frontend sources to `flowc_frontend_self.o`.
3. **Gen2.** Gen1 re-emits again; object code is compared for fixed-point equality (`cmp` of `self.o` vs. `g2.o`), with smoke tests (e.g. fixture `stage_a_sum` exiting 45).

Proposition 1 (Partial self-host fixed point). *On the Stage-A subset and frontend module set exercised by the repository’s `stage_a_self_emit_g2` scripts, consecutive self-emits produce byte-identical frontend objects under the documented toolchain. This does not yet imply that `flowc` compiles the full production language or the Python compiler sources.*

Bundle mode (`FLOWC_BUNDLE=1`) resolves relative imports and concatenates dependency C before the entry module, with optional deps-first typechecking seeded by export sets—an MVP package story inside Stage-A.

8.4 Cutover plan (architectural)

The documented plan proceeds by landing Stage-A in CI, closing the subset gap until `flowc` can compile its own sources without escape hatches, switching `./flow` to prefer the Flow driver, and only then retiring Python from the hot path [3]. Dual-run parity on tiered example corpora is the acceptance gate.

9 Implementation scale and validation posture

The repository maintains several validation layers that constrain architecture changes:

- unit and corpus tests (`./flow test`, including strict tiers);
- example status tracking across hundreds of programs;
- Stage-A roundtrip and self-emit scripts;
- optional fuzz harnesses with crash regressions folded back as parser tests;
- microbenchmarks comparing Flow-emitted C to hand-written C under matched optimization flags (historically showing parity on dense linear algebra and Mandelbrot-style kernels when both lower to equivalent C) [9].

Architecturally, these layers encode a preference for *executable honesty*: documentation and papers should track what the pipeline actually emits and runs.

10 Related work

Flow’s portable C backend sits in a lineage with languages that treat C as a portable IR (early C++, historical paths in Zig, various research transpiler languages), while its effect system relates to Eff, Koka, Frank, and Multicore OCaml’s effect handlers [11, 12, 13, 14]. Monomorphization follows the specialization tradition of C++ templates and Rust generics. MLIR as an optional path aligns with modern extensible compiler infrastructure [15]. Self-hosting bootstraps are classical (Pascal, GCC, Go, Rustc stages); Flow’s contribution is not novelty of the ladder shape but the concrete dual-host staging around a C-first systems language with domain expanders.

Relative to MATLAB/Simulink and Modelica, Flow’s dynamics surface is narrower today (LTI seed) but aims at a single deployable artifact rather than a model/code split [9]. Relative to Mojo, Flow emphasizes effects and portable C over Python-first ML runtime integration; relative to Rust, Flow trades a borrow checker for effect capabilities and a simpler default ABI story.

11 Limitations

1. **Dual host cost.** Until cutover, language bugs may require fixes in both Python and Flow-written frontends for overlapping subsets.
2. **Effect expressiveness.** Tail-resumptive handlers exclude many classical effect patterns (coroutines, non-deterministic search) without further runtime work.
3. **AD depth.** Library dual numbers and external grad tools are not yet a unified compiler AD pass.

4. **Dynamics envelope.** The shipped `dsys` analysis path is intentionally small-dimensional; larger plants still use hand-written or `stdlib` numerical code.
5. **Stage-A coverage.** Effects, full generics, and most production DSL surfaces remain outside `flowc`'s subset.

12 Conclusion

The Flow compiler is best understood as a *C-centred, dual-host* architecture: a production Python pipeline that already supports a broad language surface—including algebraic effects lowered to handler vtables, monomorphized generics, and resolve-time domain expanders—and a Flow-written Stage-A compiler that is climbing a classical self-hosting ladder toward retiring Python from the compile path. Secondary backends (MLIR, WASM via Emscripten, Metal shaders) specialize without displacing the portable default.

The architectural bets are deliberate: keep the core grammar stable; encode capabilities as ABI; expand domain syntax before parsing; specialize generics before emission; and treat self-hosting as an evidence-driven bootstrap with fixed-point checks rather than a slogan. Future work follows directly from those bets—closing the Stage-A subset gap, deepening effect runtimes beyond tail-resumption, and optionally promoting AD and richer hybrid dynamics from library/expander status into typed IR transformations—without abandoning the portable C spine that makes Flow deployable with ordinary system toolchains.

Reproducibility

The architecture described here corresponds to the open-source repository <https://github.com/floooooooooooooow/flow>. Representative commands:

```
# Production path
./flow run examples/basics/hello_world.flow
./flow compile examples/effects/showcase.flow

# Stage-A roundtrip / self-host smokes (from repo root)
./compiler/scripts/roundtrip.sh
./compiler/scripts/stage_a_self_emit_g2.sh

# Dynamics surface
./flow run examples/dynamics/ga_dsys_syntax.flow
```

Primary documentation sources mirrored in this paper: `compiler/README.md`, `docs/project/self-hosting.md`, `docs/language/dynamics-dsl.md`, `docs/effects-showcase.md`, `docs/library/autodiff.md`, and `docs/language/wasm.md`.

References

- [1] Flow project. *VISION.md — A programming language where the primary abstraction is the evolution of systems through time*. Flow repository documentation, 2026.
- [2] Flow project. *README.md* and compiler module map. <https://github.com/floooooooooooooow/flow>, 2026.
- [3] Flow project. *Self-Hosting Plan — Rewrite the Compiler in Flow*. `docs/project/self-hosting.md`, 2026.
- [4] Flow project. *Effect System Showcase: One Service, Many Worlds*. `docs/effects-showcase.md`, 2026.

- [5] Flow project. *Async via Effects*. docs/language/async-effects.md, 2026.
- [6] Flow project. *Dynamics DSL (dsys)*. docs/language/dynamics-dsl.md, 2026.
- [7] Flow project. *Autodiff*. docs/library/autodiff.md, 2026.
- [8] Flow project. *WebAssembly (WASM) target*. docs/language/wasm.md, 2026.
- [9] Flow project. *Flow vs C · Rust · Zig · Mojo*. docs/comparison.md, 2026.
- [10] Flow project. *Monomorphization pass* (src/flow/monomorphize.py), 2026.
- [11] A. Bauer and M. Pretnar. Programming with algebraic effects and handlers. *J. Log. Algebr. Methods Program.*, 84(1):– , 2015.
- [12] D. Leijen. Koka: Programming with row polymorphic effect types. *MSFP*, 2014.
- [13] S. Lindley, C. McBride, and C. McLaughlin. Do be do be do (Frank). *POPL*, 2017.
- [14] S. Dolan, S. Eliopoulos, D. Hillerström, A. Madhavapeddy, K. C. Sivaramakrishnan, and L. White. Concurrent system programming with effect handlers. *TFP*, 2017.
- [15] C. Lattner et al. MLIR: Scaling compiler infrastructure for domain languages. *CGO*, 2021.